MedSignal: Issue Analysis & Resolution Document

Issue #1 — Overpromised Real-Time Capability on a Batch Data Source

1. Issue Description

The original proposal states that MedSignal will *"process new FAERS cases within hours of submission"* using Apache Kafka and Spark Structured Streaming. This claim contains a fundamental factual error about the nature of FDA FAERS data availability.

FAERS (FDA Adverse Event Reporting System) data is released exclusively as **quarterly bulk ZIP archives** through the FDA's public data portal. Each dump contains all adverse event reports submitted in that quarter, packaged as fixed-width ASCII or XML files. There is no public submission API, no webhook notification system, no live event feed, and no mechanism by which individual case submissions are made available in real time. The pipeline between a submitted adverse event report and its appearance in the public FAERS dataset involves internal FDA processing, de-identification, quality review, and quarterly batching — a process that inherently introduces multi-month latency regardless of downstream infrastructure.

Claiming that a Kafka pipeline can process FAERS reports "within hours of submission" conflates two entirely separate systems: the FDA's internal submission intake (which is inaccessible to the public FAERS release (which is quarterly). No amount of streaming infrastructure changes the availability of the upstream data.

Why this is the most critical issue: Unlike architectural over-specification or agent design ambiguity — which are internal design choices — this is a verifiable factual claim about an external system. Any reviewer with pharmacovigilance or FDA data experience will identify it immediately, and it undermines the credibility of the entire platform's technical foundation.

2. Alternate Solutions Considered

Two straightforward alternatives exist, and both were considered before arriving at the proposed solution.

Alternative A: Remove the streaming claim entirely and reframe MedSignal as a batch system.

The simplest fix is to drop Kafka and Spark Structured Streaming altogether, process FAERS quarterly dumps as batch jobs via Airflow, and not claim any real-time capability. This is technically honest and implementable.

Why this is insufficient: It eliminates the big data engineering component that is central to the course requirements. Batch ETL over a quarterly CSV, while useful, does not demonstrate streaming pipeline design, backpressure handling, or the ability to process high-velocity data —

all of which are core learning objectives. This fix solves the honesty problem by abandoning the architectural ambition entirely, which is an overcorrection.

Alternative B: Replace FAERS with a different data source that has a genuine live API.

Sources like Twitter/X or generic health forums do have live APIs that could justify real-time streaming claims. One could pivot the primary data source away from FAERS entirely.

Why this is insufficient: FAERS is the gold standard pharmacovigilance dataset — 2M+ structured adverse event reports with drug names, reaction terms coded to MedDRA ontology, seriousness flags, and outcome data. No social media source comes close to its clinical richness or regulatory legitimacy. Abandoning FAERS to preserve a real-time claim trades scientific validity for architectural aesthetics. Furthermore, Twitter/X's API restrictions since 2023 make it practically inaccessible for academic projects at meaningful scale.

3. Proposed Solution

MedSignal adopts a **hybrid streaming architecture** that assigns each data source to accurately reflecting its actual availability, while preserving the full streaming pipeline design.

Data Source	Actual Availability	Role in MedSignal
FDA FAERS	Quarterly bulk dump	Simulated stream via Kafka replay producer
Reddit (r/pharmacy, r/ChronicPain)	Live API (rate-limited)	Genuine real-time patient signal source
PubMed/MEDLINE	Batch indexed	Static RAG corpus, refreshed quarterly
ClinicalTrials.gov	REST API, periodic updates	Batch enrichment layer

FAERS as a simulated stream: All 8GB of historical FAERS records are ingested into a Kafka producer that replays case submissions at a configurable throughput rate. This is not a workaround; it is standard practice in streaming systems research and industrial pipeline development. It allows the full Kafka → Spark Structured Streaming → NLP → Snowflake pipeline to be exercised against the complete, real FAERS dataset, including stress-testing at throughput levels that exceed what a live feed would produce. The simulation is transparent and disclosed.

Reddit as the genuine real-time signal source: This is the architecturally and scientifically more interesting claim. Published pharmacovigilance research (Freifeld et al., 2014; Nikfarjam et al.,

2015) has demonstrated that patient-reported adverse experiences on social media and health forums **precede formal FAERS submissions by two to six weeks** on average. Reddit therefore serves not merely as a supplementary data source but as a *leading indicator*: a temporally upstream signal that the platform can detect before the same event appears in official reporting channels. This is a stronger real-time claim than "we process FAERS quickly," because it asserts that MedSignal can detect signals that have not yet entered the official pipeline at all.

4. Why the Proposed Solution Is Superior

Dimension	Alternative A (Batch Only)	Alternative B (Replace FAERS)	Proposed Solution
Factual accuracy	Honest	Honest	Honest
Streaming architecture preserved	Eliminated	Yes	Yes
Clinical data quality	FAERS retained	FAERS lost	FAERS retained
Real-time component genuine	None	Dependent on API access	Reddit is live
Scientifically defensible claim	Weaker	Weaker without FAERS	Stronger — leading indicator thesis
Survives domain expert review	Yes	Questionable	Yes

The proposed solution is the only one that simultaneously preserves the streaming architecture, retains FAERS as the primary clinical dataset, introduces a genuine real-time data source, and makes a scientifically stronger claim that Reddit data is a *leading indicator* of signals not yet in the official pipeline. This reframing converts what was an architectural weakness into a research contribution.

Issue #2 — Over-Specified Technology Stack

1. Issue Description

The original proposal enumerates 13 distinct technologies: Kafka, Spark Structured Streaming, Spark NLP, Airflow, Snowflake, Pinecone, Redis, FastAPI, Streamlit, Docker Compose, AWS ECS, S3, and GitHub Actions CI/CD. While individually justifiable in a production engineering context, their collective inclusion in an academic final project creates a critical resource allocation problem: a disproportionate share of development time gets consumed by infrastructure configuration, service integration, and cloud provisioning — none of which are the graded deliverables. The course objectives center on big data engineering and intelligent systems, not cloud operations. Several technologies in the original stack are also redundant with simpler alternatives, cost real money, or solve problems that do not yet exist in the project.

The result is a proposal that sounds architecturally sophisticated but is practically at high risk of being 60% complete at submission — with a polished Kafka setup and an unfinished agent pipeline.

2. Technology-by-Technology Analysis

2.1 Message Broker — Apache Kafka ✅ Retained

Role in MedSignal: Ingests and buffers FAERS records replayed from quarterly dumps and live Reddit posts, decoupling data producers from the Spark processing layer.

Why Kafka: Kafka is the industry standard for high-throughput, fault-tolerant event streaming. Its log-based architecture means that if the downstream Spark consumer falls behind, records are not lost — they remain in the topic until consumed. This is architecturally important for MedSignal because FAERS replay will produce millions of records, and the NLP processing pipeline will be slower than ingestion. Kafka absorbs that asymmetrical cleanly.

Alternatives Considered:

Alternative	Reason Rejected
RabbitMQ	Message queue, not a log. Once a message is consumed it is gone — no ability to replay historical FAERS records, which is central to MedSignal's simulation architecture
AWS Kinesis	Functionally equivalent to Kafka but cloud-locked, costs money per shard-hour, and introduces AWS dependency that is cut from the MVP scope

Alternative	Reason Rejected
Redis Streams	Lightweight and easy to set up, but lacks Kafka's partition model, consumer group semantics, and the ability to handle the throughput of a full FAERS replay at scale
Direct file reading (no broker)	Simplest option, but eliminates the streaming architecture entirely — the pipeline becomes a batch job and the big data engineering component is lost

Verdict: Kafka is the correct and only choice for this use case. No alternative preserves both the streaming architecture and the replay capability simultaneously.

2.2 Stream Processing — Apache Spark Structured Streaming ✅ Retained

Role in MedSignal: Consumes events from Kafka topics, runs drug and symptom entity extraction at scale, computes drug-symptom co-occurrence counts, and writes structured records to PostgreSQL.

Why Spark Structured Streaming: Spark is the standard big data processing framework and is directly relevant to a course in big data systems. Structured Streaming specifically provides a unified API over both batch and streaming workloads — meaning the same NLP logic that processes a live Kafka stream can also be run over the full historical FAERS dump in batch mode for evaluation. Its DataFrame API integrates naturally with HuggingFace models via pandas UDFs. The course context alone justifies Spark — it is the expected tool.

Alternatives Considered:

Alternative	Reason Rejected
Apache Flink	Technically superior for stateful streaming with lower latency, but has a steeper learning curve, a smaller Python ecosystem, and is not a course-standard tool. The latency advantage is irrelevant for MedSignal's use case
Python + asyncio (no Spark)	Simpler to develop but eliminates the parallel, distributed processing capability that justifies the "big data" framing. Cannot scale to the full FAERS corpus without single-node bottlenecks
Pandas + multiprocessing	Same problem as above — single-machine, not a big data solution

Verdict: Spark Structured Streaming is retained. It is the right tool, the expected tool, and the one that most directly satisfies the big data engineering requirement.

2.3 NLP / Named Entity Recognition - HuggingFace BioBERT (BC5CDR) 🔄 Replaces Spark NLP

Role in MedSignal: Extracts drug names and adverse reaction terms from FAERS free-text narratives and Reddit posts, producing structured (drug, reaction) pairs for downstream signal detection.

Why HuggingFace BioBERT (BC5CDR): The en_ner_bc5cdr_md model is a BioBERT-based NER model fine-tuned specifically on the BC5CDR corpus — a gold-standard biomedical dataset of drug and disease mentions in PubMed abstracts. It produces state-of-the-art F1 scores on biomedical NER without any additional fine-tuning. Crucially, it is available via the HuggingFace transformers library, which has vastly better documentation, community support, and Python integration than Spark NLP. It can be deployed as a Spark pandas UDF, preserving Spark's distributed execution while using a better model.

Alternatives Considered:

Alternative	Reason Rejected
Spark NLP (original proposal)	Spark NLP's biomedical models (via John Snow Labs) require a paid license for the most accurate clinical models. The free tier models underperform BioBERT on drug/disease NER. Additionally, Spark NLP has a complex, non-standard API that adds significant setup overhead
spaCy + scispaCy	Excellent library with strong biomedical models (en_core_sci_lg). A close second. Rejected primarily because BioBERT-based models outperform spaCy's statistical models on drug NER benchmarks, and HuggingFace integrates more cleanly with the LLM agent layer already using the same ecosystem
Regular expressions / dictionary matching	Fast and interpretable, but brittle — drug names have thousands of brand/generic variants and misspellings in user-generated text. Recall would be unacceptably low on Reddit data
GPT-4 for NER	Accurate but expensive at scale — running GPT-4 over millions of FAERS records would cost hundreds of dollars and introduce API latency into the streaming pipeline

Verdict: HuggingFace BioBERT (BC5CDR) is the strongest combination of accuracy, cost, and integration simplicity for biomedical NER at this scale.

2.4 Data Warehouse — PostgreSQL 🔄 Replaces Snowflake

Role in MedSignal: Stores structured signal records — drug-symptom co-occurrence counts, signal scores, agent outputs, and human validation decisions — serving as the primary query backend for the Streamlit dashboard.

Why PostgreSQL: PostgreSQL is a fully-featured relational database that handles analytical queries efficiently at the data volumes MedSignal will encounter (tens of millions of FAERS records). It runs locally in Docker, costs nothing, requires no account provisioning, and is universally understood. For MedSignal's query patterns — aggregations over drug-symptom pairs, time-series signal counts, dashboard lookups — Postgres with proper indexing is more than sufficient.

Alternatives Considered:

Alternative	Reason Rejected
Snowflake (original proposal)	Cloud data warehouse designed for petabyte-scale enterprise analytics. Requires a paid account (free trial limited), external network access, and provisioning time. Adds zero academic value over Postgres for this data volume. The architectural complexity it introduces is unjustified
DuckDB	An excellent in-process analytical database with columnar storage — genuinely faster than Postgres for analytical queries. A strong alternative. Rejected because Postgres has better ecosystem support for FastAPI integration via SQLAlchemy, and DuckDB's concurrent write semantics are less mature
SQLite	Simpler than Postgres but single-writer only — cannot handle concurrent writes from Spark and reads from FastAPI simultaneously
MongoDB	Document store. Flexible schema is unnecessary here since FAERS records have a well-defined structure. Adds operational complexity without benefit

Verdict: PostgreSQL is the pragmatic, correct choice. Snowflake is an enterprise tool solving an enterprise problem that this project does not have.

2.5 Vector Store — Pinecone ✅ Retained

Role in MedSignal: Stores embeddings of 500K+ PubMed abstracts, enabling semantic similarity search for the Literature Retrieval Agent to retrieve the most relevant biomedical evidence for any drug-symptom pair.

Why Pinecone: Pinecone is a managed vector database with a clean Python SDK, a generous free tier (1 index, 100K vectors on the free plan — extendable for academic use), and sub-100ms query latency at the scale MedSignal requires. It eliminates the operational overhead of self-hosting a vector store, which is appropriate given that the RAG architecture — not the vector store infrastructure is the intellectually interesting component.

Alternatives Considered:

Alternative	Reason Rejected
Weaviate	Open-source, self-hosted vector store with strong features. A legitimate alternative. Rejected because self-hosting adds Docker complexity and operational maintenance burden compared to Pinecone's managed service
ChromaDB	Lightweight, local, easy to set up — best option for a quick prototype. Rejected because it lacks production-grade performance guarantees and its persistence model is less robust than Pinecone for 500K+ vectors
pgvector (Postgres extension)	Adds vector search directly to PostgreSQL, eliminating a separate service. Genuinely attractive for simplicity. Rejected because pgvector's approximate nearest neighbor search underperforms dedicated vector stores at 500K+ vectors, and combining OLTP and vector workloads in one Postgres instance creates resource contention
FAISS (Facebook AI Similarity Search)	Extremely fast, runs in-memory. Rejected because it has no built-in persistence — the index must be rebuilt on every restart, which is untenable for 500K embeddings

Verdict: Pinecone is retained. Its managed nature is a feature, not a shortcut — it lets development focus stay on the RAG pipeline logic rather than vector store operations.

2.6 Backend API — FastAPI ✅ Retained

Role in MedSignal: Serves as the interface layer between the Streamlit frontend, the PostgreSQL database, the Pinecone vector store, and the LLM agent pipeline. Exposes endpoints for signal queries, agent invocation, and HITL validation submission.

Why FastAPI: FastAPI is the modern standard for Python API development. Its async support is directly relevant here — LLM agent calls are I/O-bound and benefit significantly from async execution. Its automatic OpenAPI documentation generation means the API is self-documenting, which is valuable for a project where the professor may want to inspect the interface. Pydantic integration — already used for structured agent outputs — is native.

Alternatives Considered:

Alternative	Reason Rejected
Flask	Synchronous by default, requires extensions for async. Perfectly functional but strictly inferior to FastAPI for an I/O-heavy LLM application
Django	Full-stack web framework — far too heavyweight for an API layer. Brings an ORM, admin interface, and template engine that are all irrelevant here
No API layer (Streamlit calls services directly)	Simpler, but couples the frontend directly to business logic. Makes testing and future extensibility significantly harder

Verdict: FastAPI is retained. It is the right tool for a Python-native, async, LLM-integrated API layer.

2.7 Frontend — React + Tailwind 🔄 Replaces Streamlit

Role in MedSignal: Provides the pharmacovigilance dashboard — drug/symptom search, live signal timelines, AI-generated safety briefs with citation trails, and the HITL validation interface where pharmacists approve or escalate signals.

Why React + Tailwind: MedSignal's frontend has three interaction patterns that expose Streamlit's limitations: (1) a HITL validation workflow where reviewers approve, reject, or escalate signals through a structured decision interface, (2) signal timeline charts requiring fine-grained interactivity and custom styling, and (3) structured safety brief rendering with severity badges, confidence scores, and inline citation trails. All three are naturally expressed in React's component model and are awkward to implement cleanly in Streamlit. Since the team has equal comfort in both Python and JavaScript, and the backend is already FastAPI serving clean REST endpoints, React calling those endpoints is a standard, well-understood architecture. Tailwind provides rapid, consistent styling without a custom CSS overhead. The result is a dashboard that looks and behaves like a real pharmacovigilance product rather than a data science prototype.

Alternatives Considered:

Alternative	Reason Rejected
Streamlit (original proposal)	Fast to build but produces limited UI customization. The HITL validation workflow and signal timeline interactions are significantly harder to implement cleanly. Everything looks like a Streamlit app — acceptable for a quick demo, insufficient for a dashboard claiming professional pharmacovigilance use
Next.js	Everything React provides plus server-side rendering and built-in API routes. A legitimate upgrade path. Rejected because server-side rendering adds complexity without benefit for a dashboard that is primarily client-driven and already has a dedicated FastAPI backend. Retained as a future extension path
Gradio	Designed for single-input/single-output ML demos. Structurally unsuited for a multi-panel dashboard with timelines, validation workflows, and citation displays
Dash (Plotly)	Strong for data visualization. A reasonable alternative for chart-heavy dashboards. Rejected because its component model is more rigid than React for complex interaction patterns, and the team's JavaScript comfort eliminates the Python-only advantage Dash would otherwise offer

Verdict: React + Tailwind is the stronger choice given the team's JavaScript comfort, the 3-week timeline with dedicated frontend ownership, and the complexity of the HITL workflow. FastAPI's REST endpoints are consumed directly — clean separation, standard architecture, no language boundary.

2.8 Containerization — Docker Compose ✅ Retained (AWS ECS Removed)

Role in MedSignal: Defines and runs all services — Kafka, Zookeeper, Spark, PostgreSQL, Pinecone client, FastAPI, Streamlit — as isolated containers on a single machine, ensuring reproducibility across development environments.

Why Docker Compose (and not AWS ECS): Docker Compose provides the same architectural benefits as a cloud container orchestrator — service isolation, network configuration, dependency ordering, environment variable management — without requiring cloud credentials, incurring costs, or consuming development time on IAM policies and task definitions. For an academic project that runs on a professor's machine during evaluation, local Docker Compose is strictly superior. AWS ECS is retained as a documented *deployment path* — a single paragraph in the architecture section noting that each Docker service maps 1:1 to an ECS task definition — without being an active implementation target.

Alternatives Considered:

Alternative	Reason Rejected
AWS ECS (original proposal)	Cloud container orchestration. Adds real cost, requires AWS account setup, and consumes significant development time on infrastructure that is not graded
Kubernetes (k8s)	Industry-standard container orchestration. Dramatically over-engineered for a single-machine academic project
No containerization	Simplest option, but creates "works on my machine" problems during evaluation and obscures the architectural clarity of service separation

Verdict: Docker Compose is retained. AWS ECS is documented as a future deployment path only and removed from the active implementation scope.

3. Technologies Removed from MVP

Technology	Reason for Removal
Snowflake	Enterprise cost and complexity with no academic benefit over PostgreSQL
Redis	Premature optimization — add only if measured latency requires caching
Airflow	Orchestration overhead for a single quarterly refresh task — a Python script suffices
AWS ECS + S3	Cloud deployment is an extension, not a deliverable — documented as future path only
GitHub Actions CI/CD	Valuable in production, irrelevant to academic grading criteria

4. Final MVP Stack Summary

Layer	Tool	Primary Justification
Message broker	Apache Kafka	Replay capability, throughput, consumer group semantics

Layer	Tool	Primary Justification
Stream processing	Spark Structured Streaming	Course-standard, distributed NLP, batch/stream unification
NLP / NER	HuggingFace BioBERT BC5CDR	Best biomedical NER accuracy, free, Spark-compatible
Data warehouse	PostgreSQL	Sufficient for data volume, free, zero provisioning
Vector store	Pinecone	Managed, free tier, clean SDK, 500K+ vector performance
Backend API	FastAPI	Async, Pydantic-native, self-documenting
Frontend	React + Tailwind	Python-native dashboard, minimal frontend overhead
Containerization	Docker Compose	Reproducible, service-isolated, evaluator-friendly

Issue #3 — Multi-Agent Design Lacks Precision

1. Issue Description

The original proposal names three agents — Signal Detection Agent, Literature Retrieval Agent, and Clinical Severity Assessment Agent but never specifies their inputs, tools, decision logic, outputs, or handoff conditions. As written, they could be three sequential GPT-4 prompts in a Python script, which is not a multi-agent system.

A genuine agent has four properties that distinguish it from a simple LLM call: (1) it has tools it can invoke autonomously, (2) it makes decisions about whether to act, what to call, and when to stop, (3) it has a defined state it reads from and writes to, and (4) it can fail independently and trigger escalation or fallback logic. If any of the three agents lacks all four properties, it is a prompt, not an agent. The original proposal does not demonstrate that any of the three agents satisfies these criteria.

This is the issue most likely to draw hard questions during evaluation. "Multi-agent LLM pipeline" is a claim that sounds sophisticated but is straightforward to disprove if the agents cannot be described with precision.

2. Redesigned Agent Specifications

Agent 1 — Signal Detector

Property	Specification
Input	Drug-symptom co-occurrence counts from PostgreSQL, rolling 90-day window
Tools	PRR (Proportional Reporting Ratio) calculator, EBGM scorer, threshold comparator
Decision logic	If PRR > 2.0 AND case count > 3 → flag as candidate signal. If PRR > 5.0 → flag as strong signal. Below threshold → discard
Output	Ranked list of candidate drug-symptom pairs with PRR score, case count, trend direction
Handoff condition	Only pairs above threshold are passed to Agent 2 — Agent 1 acts as a gate, not a relay
Failure mode	If insufficient case count for statistical validity → mark as "insufficient data", do not escalate

Why this is genuinely an agent: It invokes statistical tools autonomously, makes a binary gate decision, and can independently determine that a signal is unactionable without involving the other agents.

Agent 2 — Evidence Retriever

Property	Specification
Input	Drug-symptom pair from Agent 1 + PRR score + case count
Tools	Pinecone semantic search, PubMed abstract fetcher, evidence summarizer
Decision logic	Runs up to 3 retrieval iterations — if first query returns low-relevance results (cosine similarity < 0.75), reformulates query with MedDRA parent term and retries. If third iteration still fails → returns "insufficient literature evidence" flag

Property	Specification
Output	Top-5 PubMed abstracts with relevance scores + 150-word synthesized evidence summary + mechanistic plausibility assessment
Handoff condition	Always passes to Agent 3, but includes a literature confidence score that Agent 3 uses to weight its severity assessment
Failure mode	Low literature confidence score propagates to Agent 3's output — signal brief explicitly states "limited published evidence"

Why this is genuinely an agent: It has iterative retrieval logic with query reformulation — it does not run one search and pass results along. The retry-with-reformulation loop is autonomous decision-making that cannot be replicated by a static prompt chain.

Agent 3 — Severity Assessor

Property	Specification
Input	Candidate signal (Agent 1) + evidence summary + literature confidence score (Agent 2) + patient outcome data from FAERS (OUTC file)
Tools	Structured output generator (Pydantic), HITL escalation trigger, confidence scorer
Decision logic	Weighs four factors: PRR magnitude, outcome severity (death/hospitalization rate), literature support strength, and novelty (is this signal already in FDA's known signal list?). Produces a severity tier: LOW / MEDIUM / HIGH / CRITICAL
Output	Structured safety brief — severity tier, confidence score, plain-language summary, full citation list, recommended action (monitor / investigate / escalate to HITL)
Handoff condition	HIGH and CRITICAL signals trigger HITL escalation — written to a human review queue in PostgreSQL. LOW and MEDIUM signals are published to the dashboard directly
Failure mode	If confidence score < 0.6 → signal is held in "pending review" state rather than published

Why this is genuinely an agent: It synthesizes inputs from two upstream agents, applies multi-factor weighted reasoning, and autonomously routes its output to different destinations based on its own assessment — two different downstream paths, not one.

3. Orchestration Framework — LangGraph

The three agents require a framework that manages typed state passing, conditional routing, retry loops, and failure handling. The choice of orchestration framework is as important as the agent design itself.

Alternatives Considered:

Framework	Reason Rejected
Plain Python functions	No state management, no retry logic, no conditional branching — this is precisely the "three prompts in a script" failure mode the proposal must avoid
LangChain sequential chain	Linear execution only — cannot handle Agent 1's gate logic (discard below threshold) or Agent 3's dual routing (HITL vs. direct publish)
CrewAI	Role-based collaborative agent framework. Well-suited for open-ended tasks where agents debate and negotiate. Wrong paradigm for a structured signal processing pipeline with strict typed handoffs
AutoGen	Designed for conversational multi-agent systems with back-and-forth dialogue between agents. Overkill and architecturally mismatched for a deterministic processing pipeline
LangGraph ✅	Explicit state machine where nodes are agents and edges are conditional handoffs. Natively handles gate conditions, retry loops, and multi-destination routing. State is typed with Pydantic throughout, ensuring structured data flows between agents

Verdict: LangGraph is the correct framework. Its state machine model maps directly onto MedSignal's agent topology it was designed for exactly this class of problem.

4. The Feature That Proves Genuine Multi-Agent Behavior

The single most convincing demonstration that MedSignal is a genuine multi-agent system — not a prompt chain — is a **feedback cycle between Agent 3 and Agent 2**. If Agent 3's confidence score falls below 0.6, it sends a request back to Agent 2 for deeper evidence retrieval before producing a final assessment:

```
Agent 1 → Agent 2 → Agent 3
          ↑       |
          └──────────────┘
```

(if confidence < 0.6,

request deeper retrieval)

This cycle cannot be implemented in a linear prompt chain. It requires LangGraph's explicit edge conditions. It is also scientifically motivated — a severity assessment with low evidence confidence should trigger additional retrieval before being published, not be suppressed silently. This loop is worth calling out explicitly in the proposal as the primary proof of genuine agentic behavior.

5. Alternate Agent Designs Considered

Alternative A — Single LLM call with a long system prompt covering all three roles

The simplest possible implementation: one GPT-4 call that receives the drug-symptom pair, retrieves literature internally via function calling, and produces a safety brief in one shot.

Why rejected: This is not a multi-agent system and does not satisfy the project's LLM architecture requirement. It also produces worse outputs — a single context window cannot maintain the iterative retrieval logic of Agent 2 or the statistical gate logic of Agent 1. Separation of concerns produces better results, not just better architecture.

Alternative B — Two agents instead of three (merge Signal Detector into statistical preprocessing)

Move the PRR calculation entirely into the Spark pipeline and start the agent system only at evidence retrieval. This simplifies the agent graph but removes the autonomous gating behavior that makes Agent 1 a genuine agent.

Why rejected: The gate decision — determining which statistical signals are worth escalating to LLM-assisted analysis — is a decision with meaningful logic and failure modes. Embedding it in Spark preprocessing hides it from the agent framework, making it harder to audit, adjust thresholds, and explain to stakeholders. Keeping it as an explicit agent node makes the decision transparent and configurable.

Issue #4 — Reddit API Practical Limitations

1. Issue Description

The original proposal lists r/pharmacy and r/ChronicPain as real-time data sources but does not address the fact that Reddit's API access was severely restricted in 2023, making the data collection strategy described in the proposal either impossible or cost-prohibitive without a concrete plan.

In June 2023, Reddit dramatically restructured its API pricing in response to the rise of LLM training data scraping. The impact on researchers and developers was significant across three dimensions. First, the free tier was capped at 100 requests per minute with a maximum of 1,000 posts per request — sufficient for light querying but severely limiting for bulk historical collection. Second, Pushshift — the third-party Reddit archive that most academic researchers used for bulk historical Reddit data — was shut down by Reddit in April 2023 and its public API was terminated. Third, the paid "Data API" tier for research use starts at several thousand dollars per month, making it non-viable for an academic project.

This means two things for MedSignal: historical data collection beyond the 1,000 most recent posts requires an alternative source, and live collection at meaningful scale requires careful rate limit management. Neither constraint is addressed in the original proposal.

2. Alternate Solutions Considered

Alternative A — Abandon Reddit entirely, use FAERS + PubMed only

Drop Reddit as a data source and rely solely on FAERS bulk data and PubMed abstracts.

Why rejected: This eliminates the genuine real-time component of the platform entirely. As established in Issue #1, Reddit is the only data source in MedSignal's stack that provides a live, continuous stream of patient-reported adverse experiences. More importantly, the leading indicator thesis — that Reddit surfaces adverse events weeks before they appear in FAERS — is one of MedSignal's strongest research contributions. Dropping Reddit to avoid an API problem is an overcorrection that weakens the scientific claim significantly.

Alternative B — Use the paid Reddit Data API

Apply for Reddit's official research data access program and pay for elevated API access.

Why rejected: Cost is prohibitive for an academic project — Reddit's commercial API tiers are priced for enterprise use. The application process also takes several weeks and approval is not guaranteed, making it an unreliable dependency for a 3-week implementation timeline.

Alternative C — Scrape Reddit directly via web scraping

Bypass the API entirely using a scraping library such as BeautifulSoup or Playwright.

Why rejected: This violates Reddit's Terms of Service explicitly. Beyond the ethical problem, it creates a brittle data collection layer that breaks whenever Reddit changes its HTML structure — a maintenance burden incompatible with a 3-week timeline.

3. Proposed Solution — Three-Layer Reddit Strategy

Rather than a single Reddit collection approach, MedSignal uses three complementary mechanisms that together provide both historical depth and live coverage within free API constraints.

Layer 1 — Academic Torrent Archive (Historical Corpus) The Reddit academic community has maintained publicly available compressed archives of historical Reddit posts, known as Pushshift dumps, which were collected before the API shutdown. These are legally redistributed for academic use and cover posts up to early 2023. For MedSignal, the relevant subreddit archives — r/pharmacy, r/ChronicPain, r/AskDocs, r/druginteractions — total approximately 2–4GB compressed and provide years of historical patient-reported adverse experiences for retrospective analysis and golden set evaluation.

Layer 2 — PRAW Live Collection (Real-Time Stream) PRAW (Python Reddit API Wrapper) provides a clean interface to Reddit's free API tier. Using subreddit.stream.submissions(), MedSignal continuously collects new posts from target subreddits in real time within free rate limits. At typical posting volumes in health subreddits, the free tier is sufficient — r/ChronicPain averages 50–100 new posts per day, well within the free API ceiling. A PRAW producer publishes new posts directly to a dedicated Kafka topic as they arrive, feeding the same Spark NLP pipeline as FAERS records.

Layer 3 — Targeted Keyword Search (Signal-Driven Collection) When Agent 1 identifies a candidate drug-symptom signal, a targeted Reddit search is triggered via the API — searching for recent posts mentioning the specific drug and reaction combination. This inverts the collection logic: instead of collecting everything and filtering, collection is triggered by signal detection. This is both more efficient and more scientifically interesting — it grounds each signal brief in contemporaneous patient discourse at the moment of signal identification.

4. Why the Proposed Solution Is Scientifically Stronger

The three-layer strategy is not merely a workaround for API restrictions — it produces a better research design than unrestricted API access would have enabled.

Dimension	Original Proposal	Proposed Solution
Historical coverage	Implied but unplanned	Explicit — Pushshift archives through 2023
Real-time coverage	Assumed free API access	PRAW stream within verified free tier limits
Collection efficiency	Bulk collection of all posts	Signal-driven targeted collection in Layer 3

Dimension	Original Proposal	Proposed Solution
Leading indicator thesis	Stated but untestable	Directly testable — Reddit post dates vs. FAERS submission dates
API cost	Unaddressed	Zero — free tier + public academic archive

The leading indicator thesis in particular becomes directly testable with this design: for signals in the golden set evaluation, the date of first Reddit mention can be measured against the date of first FAERS report for the same drug-reaction pair, quantifying the lead time in days. This transforms a narrative claim into a measurable finding — and a measurable finding is a research contribution.

Issue #5 — GPT-4 as Automated Evaluator Creates Circularity

1. Issue Description

The original proposal uses GPT-4 to automatically evaluate the quality of outputs generated by a GPT-4-powered agent pipeline — a circular evaluation design where the model being assessed and the model doing the assessing are identical, making the evaluation scientifically meaningless.

Evaluation circularity is a well-documented failure mode in LLM system design. When GPT-4 evaluates GPT-4 outputs, three systematic biases emerge. First, self-preference bias — research has demonstrated that LLMs consistently rate their own outputs higher than outputs from other models, even when blind to authorship. A GPT-4 evaluator will systematically favor GPT-4-generated safety briefs regardless of their actual clinical accuracy. Second, shared failure modes — if GPT-4 has a systematic blind spot, such as consistently overstating mechanistic plausibility for a class of drug interactions, both the generator and the evaluator share that blind spot, causing confidently wrong outputs to score as correct. Third, absence of ground truth — scientific evaluation requires an external reference point that exists independently of the system being evaluated. A GPT-4 evaluator has no such anchor.

A reviewer will identify the circularity immediately. More critically, any results produced by a circular evaluation are uninterpretable — no claims about system quality can be made based on them.

2. Alternate Solutions Considered

Alternative A — Human expert evaluation only

Replace GPT-4 evaluation entirely with human pharmacist or physician review of all 50 signals in the golden set.

Why rejected: Expert clinical review is the gold standard but is not scalable within a 3-week timeline for 50 signals. Recruiting qualified reviewers, designing a review instrument, and collecting annotations takes weeks. Human evaluation is retained as a component but cannot be the sole evaluation method.

Alternative B — Switch to a different LLM as evaluator

Use Claude or Gemini as the evaluator instead of GPT-4.

Why rejected: This reduces but does not eliminate the circularity problem. The evaluator is still an LLM with shared training data distributions and similar failure modes to GPT-4. Differences in scores may reflect model preference disagreements rather than actual output quality differences — introducing a new confound without resolving the fundamental issue.

Alternative C — Remove automated evaluation entirely

Rely only on the retrospective signal detection comparison and drop automated output quality evaluation.

Why rejected: The retrospective detection comparison evaluates whether MedSignal detects signals, but not how well it explains them. A system that detects the right signals but produces incoherent safety briefs is not a useful pharmacovigilance tool. Output quality evaluation is a necessary second dimension and cannot be dropped entirely.

3. Proposed Solution — Three-Layer Evaluation Framework

MedSignal replaces the single GPT-4 evaluator with a three-layer evaluation framework where each layer measures a distinct dimension of system quality against an independent, non-circular ground truth.

Layer 1 — Retrospective Signal Detection (Primary Evaluation)

What it measures: Whether MedSignal detects known drug safety signals, and how early relative to official FDA recognition.

Method: 10 historical drug safety signals with known FDA communication dates serve as the golden set. For each signal, MedSignal is run on FAERS data from the quarters preceding the FDA's official communication. The earliest date at which Agent 1 flags the signal above threshold is recorded. Detection lead time — the gap between MedSignal's detection date and the FDA's official date — is the primary metric.

Why non-circular: The ground truth is the FDA's official communication date — an external, human-determined reference that exists entirely independently of MedSignal or any LLM.

Metrics: Detection lead time in days, precision (fraction of MedSignal's flagged signals that are confirmed real), recall (fraction of known signals MedSignal successfully detects).

Example golden set signals:

Drug	Reaction	FDA Communication Year
Rofecoxib (Vioxx)	Cardiovascular events	2004
Metoclopramide	Tardive dyskinesia	2009
Azithromycin	QT prolongation / cardiac death	2013
Fluoroquinolones	Aortic aneurysm	2018
Varenicline (Chantix)	Neuropsychiatric events	2009

Layer 2 — NER Extraction Quality (Component Evaluation)

What it measures: Whether the Spark + BioBERT pipeline correctly extracts drug and symptom entities from FAERS free-text narratives.

Method: The n2c2 (formerly i2b2) shared task datasets provide human-annotated clinical NER benchmarks with gold-standard drug and adverse event entity labels. BioBERT extraction outputs are compared against these annotations using standard NER metrics.

Why non-circular: Ground truth annotations are produced by human clinical annotators — no LLM is involved in their creation.

Metrics: Precision, recall, and F1 score on drug entity extraction and adverse reaction extraction evaluated separately.

Layer 3 — RAG Retrieval Quality (Component Evaluation)

What it measures: Whether Agent 2 retrieves genuinely relevant PubMed literature for a given drug-symptom pair.

Method: For 20 drug-symptom pairs with well-established literature, a pharmacist annotator identifies the 5 most relevant PubMed abstracts as ground truth. MedSignal's retrieval results are compared against this annotation set using standard information retrieval metrics.

Why non-circular: Relevance judgments are made by a human pharmacist, not GPT-4.

Metrics: Mean Reciprocal Rank (MRR), Precision@5, NDCG@5.

GPT-4 Evaluation — Retained for One Appropriate Use Case

GPT-4 evaluation is not eliminated entirely. It is retained for one narrow, non-circular use case: evaluating the linguistic quality and coherence of safety brief text — not its clinical accuracy. Fluency, clarity, and citation formatting are dimensions where LLM evaluation introduces no circularity, since there is no clinical ground truth to be biased against. This is disclosed explicitly as a secondary, supplementary metric with acknowledged limitations.

4. Evaluation Framework Summary

Layer	What It Measures	Ground Truth Source	Primary Metrics
1 — Signal Detection	Does MedSignal detect real signals? How early?	FDA official communication dates	Detection lead time, precision, recall
2 — NER Quality	Does BioBERT extract entities correctly?	n2c2 human annotations	F1, precision, recall
3 — RAG Retrieval	Does Agent 2 retrieve relevant literature?	Pharmacist relevance judgments	MRR, Precision@5, NDCG@5
GPT-4 (secondary)	Are safety briefs well-written?	N/A — linguistic quality only	Fluency, coherence scores

This framework evaluates MedSignal end-to-end (Layer 1), at the component level (Layers 2 and 3), and at the output quality level (GPT-4 secondary) — each with an independent, non-circular ground truth. No single evaluation dimension depends on GPT-4 judging GPT-4.

Issue #6 — Missing System Architecture Diagram and Project Timeline

1. Issue Description

The original proposal describes all system components in prose but never visualizes how data flows through the platform end-to-end, and provides no implementation timeline. These are not cosmetic omissions — they are structural gaps that create two distinct problems.

The absence of an architecture diagram means the proposal describes a system without proving it is coherent. A list of technologies does not demonstrate that the team understands how Kafka connects to Spark, how Spark output reaches the agent pipeline, how the agents interact with storage, or how the frontend consumes backend results. Without a diagram, a reviewer cannot

verify that the architecture is internally consistent — they must take it on faith. In a technically evaluated project, that is an unacceptable dependency.

The absence of a timeline means the proposal has no answer to the most basic project management question: what gets built when, and by whom? A proposal without a timeline is a vision document, not a project plan. It also makes it impossible to assess whether the scope is achievable within the available time which, given the risks identified in Issues #1 through #5, is a critical question.

2. System Architecture

The full system architecture is organized into six horizontal layers, each with a distinct responsibility:

Layer 1 — Data Sources: Four inputs feed the platform. FDA FAERS quarterly bulk dumps and Reddit posts via PRAW stream into Kafka. ClinicalTrials.gov updates are pulled via REST API into Kafka. PubMed abstracts bypass Kafka entirely — they are processed through a separate embedding pipeline that writes directly to Pinecone, since they serve as a static RAG corpus rather than a streaming event source.

Layer 2 — Ingestion (Apache Kafka): A single Kafka cluster with three named topics — faers-events, reddit-posts, and clinicaltrials-updates — decouples all producers from the downstream processing layer. FAERS records are replayed from historical bulk files through a Python Kafka producer at configurable throughput. Reddit posts arrive via a PRAW live stream producer. This layer absorbs ingestion bursts without backpressure reaching the NLP pipeline.

Layer 3 — Stream Processing (Spark Structured Streaming): Spark consumes all three Kafka topics, runs HuggingFace BioBERT BC5CDR named entity recognition over free-text narratives to extract drug and symptom mentions, computes drug-symptom co-occurrence counts over rolling 90-day windows, and calculates PRR and EBGM disproportionality scores. Structured output is written to PostgreSQL. This layer is the primary big data engineering component.

Layer 4 — Agent Pipeline (LangGraph): Three agents operate as nodes in a LangGraph state machine. Agent 1 (Signal Detector) applies a PRR threshold gate — pairs below threshold are discarded, pairs above threshold are passed to Agent 2. Agent 2 (Evidence Retriever) performs iterative semantic search over Pinecone with up to three query reformulation attempts. Agent 3 (Severity Assessor) synthesizes signal statistics, retrieved evidence, and patient outcome data into a structured Pydantic safety brief. A feedback edge from Agent 3 back to Agent 2 is triggered when output confidence falls below 0.6, requesting deeper retrieval before a final brief is produced.

Layer 5 — Storage: Three services with distinct roles. PostgreSQL stores structured signal records, agent-generated safety briefs, co-occurrence counts, and the HITL review queue. Pinecone stores 500K PubMed abstract embeddings and serves semantic search queries from Agent 2. The GPT-4 API is invoked by all three agents for reasoning and text generation.

Layer 6 — Presentation: A FastAPI backend exposes REST endpoints for signal queries, agent invocation, and HITL decision submission. A React + Tailwind frontend renders the pharmacovigilance dashboard — signal timelines, safety briefs with inline citations, severity badges, and the human validation interface. All services run under Docker Compose for local reproducibility.